

C++ 自学手册：第 6 章 分支语句和逻辑运算符

一、核心知识框架

本章围绕程序的**决策能力**展开，主要内容包括：

- 分支语句：if、if-else、if-else if-else
- 逻辑运算符：&&（与）、||（或）、!（非）
- 字符处理：cctype 函数库
- 条件运算符：?:
- 多分支选择：switch 语句
- 循环控制：break 和 continue
- 健壮输入：处理数字输入的循环
- 文件操作：基本文本文件的读写

二、分支语句（if 系列）

1. 基本 if 语句

作用：根据条件执行特定代码块，条件为true时执行，否则跳过。

语法：

```
if (测试条件) {  
    // 条件为true时执行的语句块  
}
```

示例：统计输入中的空格数

```
char ch;  
int spaces = 0;  
cin.get(ch);
```

```
while (ch != '!') { // 遇到句点结束
    if (ch == ' ') // 若为空格则计数
        spaces++;
    cin.get(ch);
}
cout << "空格数: " << spaces << endl;
```

注意：

- 测试条件会被转换为bool类型（0为false，非0为true）。
- 若语句块仅含一条语句，大括号可省略，但建议始终添加以避免逻辑错误。

2. if-else 语句

作用：二选一执行，条件为true时执行if块，否则执行else块。

语法：

```
if (测试条件) {
    // 条件为true时执行
} else {
    // 条件为false时执行
}
```

示例：判断数字是否为偶数

```
int num;
cin >> num;
if (num % 2 == 0) {
    cout << "偶数" << endl;
} else {
    cout << "奇数" << endl;
}
```

3. if-else if-else 语句

作用：多条件分支，依次判断条件，执行第一个满足条件的分支。

语法：

```
if (条件1) {  
    // 条件1为true时执行  
} else if (条件2) {  
    // 条件2为true时执行  
} else {  
    // 所有条件都为false时执行  
}
```

示例：猜数字游戏

```
const int target = 27;  
int guess;  
cin >> guess;  
if (guess < target) {  
    cout << "太小了" << endl;  
} else if (guess > target) {  
    cout << "太大了" << endl;  
} else {  
    cout << "猜对了！" << endl;  
}
```

三、逻辑运算符

用于组合或修改条件，处理多条件判断。

运算符	名称	语法	含义	短路特性
&&	逻辑与	expr1 && expr2	两者都为 true 时结果为 true	若 expr1 为 false，不执行 expr2
、		、	逻辑或	`expr1
!	逻辑非	!expr	对 expr 的真值取反	无

1. 逻辑与 (&&)

示例：判断年龄在 18~34 岁之间

```
int age = 25;
if (age > 17 && age < 35) { // 等价于(age>17) && (age<35)
    cout << "青年" << endl;
}
```

2. 逻辑或 (||)

示例：判断输入为 'y' 或 'Y'

```
char ch;
cin >> ch;
if (ch == 'y' || ch == 'Y') { // 接受大小写
    cout << "继续操作" << endl;
}
```

3. 逻辑非 (!)

示例：判断字符串是否相同（结合 strcmp）

```
#include <cstring>
char s1[] = "hello", s2[] = "world";
if (!strcmp(s1, s2)) { // strcmp返回0时表示相同，!0为true
    cout << "字符串相同" << endl;
}
```

4. 优先级与注意事项

- 优先级：! > 关系运算符 > && > || > 赋值运算符。
- 避免连续比较（如 `17 < age < 35` 是错误的，应改为 `age > 17 && age < 35`）。
- 替代表示：`&&`→and，`||`→or，`!`→not（需包含头文件 `iso646.h`，C++ 可直接使用）。

四、cctype 字符函数库

提供一组用于字符分类和转换的函数，简化字符处理（需包含头文件<cctype>）。

函数	功能	示例 (ch = 'A')
isalpha(ch)	判断是否为字母 (a-z 或 A-Z)	true
isdigit(ch)	判断是否为数字 (0-9)	false
isspace(ch)	判断是否为空白 (空格、换行、制表符等)	false
ispunct(ch)	判断是否为标点符号	false
tolower(ch)	转换为小写 (非字母则不变)	'a'
toupper(ch)	转换为大写 (非字母则不变)	'A' (本身是大写)

示例：统计文本中的字符类型

```
#include <cctype>
char ch;
int letters = 0, digits = 0;
while (cin.get(ch) && ch != '@') {
    if (isalpha(ch)) letters++;
    else if (isdigit(ch)) digits++;
}
cout << "字母: " << letters << ", 数字: " << digits << endl;
```

五、条件运算符 (?:)

作用：简化简单的二选一判断，是if-else的紧凑形式。

语法：

表达式1 ? 表达式2 : 表达式3; // 若表达式1为true, 结果为表达式2, 否则为表达式3

示例：求两数中的较大值

```
int a = 5, b = 8;
int max = (a > b) ? a : b; // 等价于if(a>b) max=a; else max=b;
cout << "最大值: " << max << endl; // 输出8
```

注意：

- 适合简单逻辑, 复杂条件建议用if-else以保证可读性。
- 可嵌套使用, 但嵌套过多会降低可读性。

六、switch 语句

作用：多分支选择, 根据整数表达式的值匹配case标签执行对应代码。

语法：

```
switch (整数表达式) {
    case 常量1: // 标签必须是整数常量 (如1、'a'、枚举值)
        语句块1;
        break; // 跳出switch
    case 常量2:
        语句块2;
        break;
    default: // 所有case不匹配时执行 (可选)
        语句块3;
}
```

示例：菜单选择

```
int choice;
cout << "1. 加法 2. 减法 3. 退出" << endl;
cin >> choice;
switch (choice) {
    case 1:
        cout << "执行加法" << endl;
```

```
break;
case 2:
cout << "执行减法" << endl;
break;
case 3:
cout << "退出" << endl;
break;
default:
cout << "无效选择" << endl;
}
```

关键注意事项：

- `case` 标签必须是**整数常量**（不能是变量）。
- `break` 的作用：阻止代码“贯穿”到下一个 `case`（若省略，会继续执行后续所有 `case`）。
- 与 `if-else` 的选择：选项 ≥ 3 个时优先用 `switch`，更高效且清晰。

七、`break` 与 `continue`

1. `break` 语句

- **作用**：立即跳出当前循环或 `switch` 语句，执行后续代码。
- **适用场景**：循环中提前终止（如遇到特定条件）、`switch` 中避免贯穿。

示例：遇到句点则停止输出

```
char line[80];
cin.getline(line, 80);
for (int i = 0; line[i] != '\0'; i++) {
    if (line[i] == '.') break; // 遇到句点则跳出循环
    cout << line[i];
}
```

2. `continue` 语句

- **作用**：跳过当前循环体中剩余代码，直接开始下一次循环。

- **适用场景：**过滤不需要的元素，仅处理满足条件的数据。

示例：只统计空格数

```
char line[80];
cin.getline(line, 80);
int spaces = 0;
for (int i = 0; line[i] != '\0'; i++) {
    if (line[i] != ' ') continue; // 非空格则跳过计数
    spaces++;
}
cout << "空格数: " << spaces << endl;
```

八、读取数字的循环（健壮输入处理）

处理用户输入时，需应对非数字输入的情况，确保程序不崩溃。

核心步骤：

1. 检测输入错误（cin转换失败时返回false）。
2. 重置输入流状态（cin.clear()）。
3. 清除错误输入（cin.ignore()）。
4. 提示重新输入。

示例：读取 5 个高尔夫分数（处理非数字输入）

```
const int Max = 5;
int golf[Max];
for (int i = 0; i < Max; i++) {
    cout << "第" << i+1 << "轮分数: ";
    while (!(cin >> golf[i])) { // 若输入非数字
        cin.clear(); // 重置错误状态
        cin.ignore(1000, '\n'); // 清除当前行错误输入
        cout << "请输入数字: ";
    }
}
```


九、基本文件输入 / 输出

1. 文件写入 (ofstream)

步骤：

- 包含头文件<fstream>。
- 创建ofstream对象，关联文件（open()）。
- 像cout一样写入数据。
- 关闭文件（close()）。

示例：将数据写入文件

```
#include <fstream>
ofstream outFile;
outFile.open("data.txt"); // 打开文件（不存在则创建）
if (!outFile.is_open()) { // 检查文件是否打开成功
    cerr << "文件打开失败" << endl;
    exit(EXIT_FAILURE);
}
outFile << "姓名：张三" << endl; // 写入数据
outFile << "年龄：20" << endl;
outFile.close(); // 关闭文件
```

2. 文件读取 (ifstream)

步骤：

- 包含头文件<fstream>。
- 创建ifstream对象，关联文件（open()）。
- 像cin一样读取数据。
- 关闭文件（close()）。

示例：从文件读取数字并计算总和

```
#include <fstream>
ifstream inFile;
inFile.open("scores.txt");
if (!inFile.is_open()) {
    cerr << "文件打开失败" << endl;
    exit(EXIT_FAILURE);
}
double sum = 0, num;
int count = 0;
while (inFile >> num) { // 读取所有数字
    sum += num;
    count++;
}
cout << "总和: " << sum << ", 平均值: " << sum/count << endl;
inFile.close();
```

十、重点警告与自查清单

1. **if语句**：避免将`==`误写为`=`，可写成`3 == x`（编译器会报错）。
2. **逻辑运算符**：注意短路特性（`expr1 && expr2`中若`expr1`为`false`，`expr2`不执行）。
3. **switch语句**：每个`case`后需加`break`，否则会贯穿执行。
4. **文件操作**：打开文件后务必检查是否成功，避免空指针操作。
5. **输入处理**：非数字输入后需用`clear()`和`ignore()`重置流状态，否则循环会卡死。

十一、自测练习

1. 用`if-else if-else`判断学生成绩等级（90+ 为 A，80-89 为 B，否则为 C）。
2. 使用`&&`和`isalpha`函数统计文本中大写字母的数量。
3. 用`switch`语句实现简单计算器（支持`+`、`-`、`*`、`/`）。
4. 编写程序，读取用户输入直到`q`，用`continue`跳过空格，统计总字符数。
5. 将 10 个随机数写入文件，再读取并计算它们的平方和。

通过本章学习，可掌握程序的分支决策和逻辑控制，结合文件操作实现数据的持久化存储，为复杂程序设计奠定基础。

（注：文档部分内容可能由 AI 生成）